

DDD-Enforcer: An Empirical Study of LLM-Powered Domain-Driven Design Enforcement in Microservice Systems

Ahmet Baran Dincoguz · Ali Kendir ·
Murat Karakaya

Received: date / Accepted: date

Abstract Domain-Driven Design (DDD) provides a principled approach to structuring software around business domains, yet maintaining DDD compliance in evolving codebases remains a persistent challenge. Traditional static analysis tools enforce syntactic and structural rules but lack the semantic understanding necessary to detect domain-concept-level violations such as ubiquitous language misuse or bounded context boundary crossings. Prior LLM-assisted DDD work has focused on supporting up-front *design* activities (event storming, bounded-context discovery) rather than the continuous *enforcement* of an existing domain model against an evolving codebase. In this paper, we present DDD-Enforcer, a proof-of-concept framework for LLM-powered DDD enforcement that automatically extracts a machine-readable rulebook from a Software Requirements Specification (SRS) and uses it to flag DDD violations in source code through an IDE integration. We instantiate the framework with a four-stage multi-agent pipeline (Scout, Architect, Specialist, Synthesizer) that produces the rulebook, an AST-guided validator that checks code against it, and a VS Code extension that surfaces violations to the developer. The rulebook is a durable artifact in its own right: beyond enabling automated enforcement, it serves as a machine-checked onboarding document for new developers joining a project. Because expert-annotated ground truth at the scale we require is infeasible to obtain, we evaluate the framework using *LLM-Assisted Human Evaluation*: a strictly stronger Judge LLM clas-

A. B. Dincoguz
Department of Software Engineering, TED University, Ankara, Türkiye
E-mail: baran.dincoguz@tedu.edu.tr

A. Kendir
Department of Software Engineering, TED University, Ankara, Türkiye
E-mail: ali.kendir@tedu.edu.tr

M. Karakaya
Department of Software Engineering, TED University, Ankara, Türkiye
E-mail: murat.karakaya@tedu.edu.tr

sifies each reported violation against a detailed rubric, and the authors audit a sample of its verdicts. We compare three pipeline variants (naive single-call, retrieval-augmented single-call, and the multi-agent pipeline), four LLM providers (Google Gemini, OpenAI ChatGPT, Anthropic Claude, and an open-source model), and three SRS domains, and we additionally measure recall on codebases into which we deliberately seed DDD violations. Our goal is not to prove that DDD-Enforcer is a finished product, but to demonstrate that LLM-enforced DDD is feasible at today’s capability level and to provide a reusable framework that other researchers can extend.

Keywords Domain-Driven Design · Large Language Models · Microservices · Software Architecture · Static Analysis · Multi-Agent Systems · LLM-as-a-Judge

1 Introduction

Domain-Driven Design (DDD) [1] provides a systematic methodology for structuring complex software systems around business domain concepts. Central to DDD are the notions of *ubiquitous language*—a shared vocabulary between domain experts and developers—and *bounded contexts*, which define clear conceptual boundaries within which a domain model is consistent and applicable [2]. These principles are particularly relevant in microservice architectures, where each service ideally corresponds to a bounded context with its own domain model [3].

Despite the widespread adoption of DDD principles in microservice design [6], maintaining DDD compliance as codebases evolve remains a significant challenge. *Domain model degradation*—the gradual erosion of DDD principles through development iterations—manifests in several forms: violations of the ubiquitous language (e.g., introducing synonyms such as “Client” where the domain model defines “Customer”), breaches of bounded context boundaries (e.g., one service directly importing entities from another without declared dependencies), and the emergence of anti-patterns such as the Anaemic Domain Model [4].

Existing architecture enforcement tools such as ArchUnit [7] and NDepend [8] enable developers to define and verify structural constraints through unit test assertions and static analysis queries. However, these tools operate at the syntactic and structural level—they can verify that a class resides in a specific package or that a dependency direction is correct, but they cannot determine whether a naming choice violates the ubiquitous language or whether a service’s responsibilities have drifted beyond its bounded context. This semantic gap leaves a significant class of DDD violations undetectable by current tooling.

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in code understanding, program comprehension, and software engineering tasks [9]. LLMs possess the semantic reasoning ability to understand domain concepts, detect naming inconsistencies, and reason about

architectural boundaries in ways that rule-based tools cannot. A small but growing body of work has begun applying LLMs to DDD itself [17,?], but the published efforts concentrate on the *design* side of DDD—using LLMs as collaborative sparring partners for event storming, ubiquitous-language establishment, and bounded-context discovery. The complementary problem of *continuously enforcing* an already-established domain model against a growing codebase has, to the best of our knowledge, not been systematically studied. This is the gap that this paper aims to open up.

In this paper, we present DDD-Enforcer, a *proof-of-concept framework* for LLM-powered DDD enforcement. Rather than arguing that a single finished system is production-ready, our aim is to demonstrate the feasibility of LLM-enforced DDD at today’s capability level and to provide a reusable scaffolding that other researchers can extend, swap components out of, and re-evaluate as LLMs improve. We instantiate the framework with a four-stage multi-agent pipeline (Scout, Architect, Specialist, Synthesizer) that produces a structured rulebook from an SRS document, an AST-guided validator that checks source code against that rulebook, and a VS Code extension that surfaces violations to the developer. The rulebook is a first-class artifact in its own right: because it is derived from the SRS and is machine-checkable, it doubles as an onboarding document for developers newly assigned to the project. We extend our earlier conference publication [24] with a substantially expanded empirical study organized around the following research questions:

- RQ1** *Pipeline comparison.* Holding the LLM fixed, how do alternative pipeline architectures (naive single-call, retrieval-augmented single-call, and our multi-agent pipeline) compare in rulebook quality and DDD violation detection?
- RQ2** *Model comparison.* Using the best pipeline from RQ1, how do four LLM providers (Google Gemini, OpenAI ChatGPT, Anthropic Claude, and an open-source local model) compare on detection accuracy, latency, and cost?
- RQ3** *Cross-domain generalization.* Using the best pipeline and model combination, how does the framework perform across three SRS domains drawn from distinct application areas?
- RQ4** *Synthetic-violation recognition.* On codebases into which we have deliberately seeded DDD violations, how well does the framework recover the planted violations?

Our key contributions are:

- A **framework for LLM-powered DDD enforcement** that separates rulebook extraction from violation detection and is designed so that each component can be replaced independently.
- A concrete **four-stage multi-agent instantiation** of the rulebook-extraction component (Scout, Architect, Specialist, Synthesizer), compared head-to-head against naive and retrieval-augmented alternatives.
- An **LLM-Assisted Human Evaluation protocol** that substitutes a strictly stronger Judge LLM plus author audit for expert-annotated ground

truth, providing a practical evaluation methodology for proof-of-concept studies where expert annotation is infeasible.

- An **empirical snapshot** across three pipelines, four LLM providers, and three SRS domains that demonstrates feasibility without claiming optimality.
- An **open release** of the framework, configuration files, rubric prompts, and raw results, so that other researchers can swap in new pipelines, models, or domains and re-run the study.

The remainder of this paper is organized as follows. Section 2 reviews the background and related work. Section 3 describes the DDD-Enforcer framework and its current instantiation. Section 4 presents the experimental design, including the LLM-Assisted Human Evaluation protocol. Sections 5 through 8 report our results organized by research question. Section 9 discusses findings, implications, and threats to validity, and Section 10 concludes the paper.

2 Background and Related Work

2.1 Domain-Driven Design Principles and Practices

Domain-Driven Design, introduced by Evans [1] and further elaborated by Vernon [2], provides a set of strategic and tactical patterns for modeling complex software around business domains. *Strategic patterns* include bounded contexts (defining the applicability boundary of a domain model), context maps (documenting relationships between bounded contexts), and the ubiquitous language (a shared structured vocabulary connecting domain activities with code). *Tactical patterns* include entities, value objects, aggregates, domain events, repositories, and domain services—building blocks for expressing domain logic within a bounded context.

In microservice architectures, DDD’s strategic patterns provide natural guidance for service decomposition: each microservice ideally aligns with a bounded context, encapsulating a coherent slice of domain logic [3]. However, applying DDD in practice is challenging. Ozkan et al. [4] conducted an industrial action research study in which they refactored a Java/Spring backend system using DDD principles. Their findings revealed that while DDD significantly improved modularity, testability, and reusability (achieving an mTAM score of 84/100), the approach introduced substantial cognitive load—engineers without prior DDD experience found the abstractions difficult to grasp. Importantly, their study was conducted entirely manually: domain experts identified bounded contexts, domain services, and domain objects through focus groups and expert judgment. Our work aims to automate this process through LLM-powered analysis.

[PLACEHOLDER: Add 2–3 more DDD-related references: studies on DDD adoption challenges, DDD in microservices decomposition (Mazlami et al. 2017, Kapferer and Zimmermann 2020), DDD anti-patterns.]

2.2 Architecture Enforcement and Static Analysis

Several tools enable developers to enforce architectural constraints through automated checks. ArchUnit [7] allows Java developers to express architecture rules as unit tests, verifying package dependencies, class naming conventions, and layer access restrictions. NDepend [8] provides similar capabilities for .NET through LINQ-based queries and comprehensive code metrics. NetArchTest offers comparable functionality for .NET Core.

While these tools are effective for structural constraints, they share a fundamental limitation: rules must be manually defined and maintained, and the tools lack semantic understanding of domain concepts. ArchUnit can verify that a class in the `order` package does not import from the `inventory` package, but it cannot determine whether a class named `ClientProfile` violates the ubiquitous language that defines the concept as `CustomerProfile`. This semantic gap is precisely what LLM-powered analysis can address.

[PLACEHOLDER: Add references to more architecture enforcement literature: architectural erosion studies, architecture conformance checking (Knodel and Popescu 2007, Passos et al. 2010).]

2.3 Microservice Quality Analysis

Research on automated quality analysis of microservice systems has explored various dimensions. Abdelfattah et al. [5] proposed a Component-based Semantic Clone (CSC) detection method for microservices, using Component Call Graphs and logistic regression to identify functionally similar code across services. Their approach achieved an F1-score of 0.813 on the TrainTicket benchmark and demonstrated that semantic clones (code that is functionally equivalent but syntactically different) represent a distinct phenomenon from syntactic clones. However, their method relies on handcrafted features (method names, types, HTTP methods) combined with WordNet-based similarity and supervised learning on manually labeled datasets of over 27,000 pairs.

Our work addresses a complementary problem: while semantic clone detection identifies *symptoms* of poor microservice decomposition (duplicated functionality), DDD violation detection targets the *root causes* (misaligned bounded contexts, ubiquitous language drift). Furthermore, our LLM-based approach requires no manual feature engineering or labeled training data, leveraging the LLM’s pre-trained understanding of domain semantics.

[PLACEHOLDER: Add references to: microservice anti-pattern detection, service decomposition quality metrics, API analysis in microservices.]

2.4 LLMs for Software Engineering

Large Language Models have been applied to a wide range of software engineering tasks, including code generation [11], bug detection [12], code review [13],

and program comprehension [14]. Recent surveys [9, ?] provide comprehensive overviews of the landscape.

In the domain of software architecture, LLMs have been explored for architecture recovery, design pattern identification, and code smell detection. **[PLACEHOLDER: Add 3–4 specific references on LLM-based architecture analysis.]** However, the application of LLMs specifically to Domain-Driven Design enforcement remains largely unexplored. To the best of our knowledge, DDD-Enforcer is among the first systems to combine LLM-powered semantic analysis with AST inspection for real-time DDD compliance checking.

AI coding assistants such as GitHub Copilot and Amazon CodeWhisperer [15] focus primarily on code generation and completion rather than architectural compliance. While these tools can suggest code that follows certain patterns, they do not actively enforce domain design principles or detect violations against a project-specific domain model.

A parallel strand of work has begun treating LLMs not only as code generators but also as *evaluators* of code and software artifacts. Recent empirical studies of *LLM-as-a-Judge* methodologies in software engineering [20, 21, 22] report that output-based LLM judges achieve Pearson correlations around 0.8 with human experts on code-related evaluation tasks, significantly outperforming traditional lexical metrics. These results suggest that a sufficiently strong LLM, guided by a detailed rubric, can approximate human judgment for many SE evaluation tasks. Our evaluation methodology (Section 4.5) builds directly on this line of work.

2.5 LLMs for Domain-Driven Design

A small body of work has started exploring how LLMs can assist with DDD. The prompting-framework proposal of [17] decomposes DDD into five sequential steps (ubiquitous-language establishment, event-storming simulation, bounded-context identification, aggregate design, technical-architecture mapping) and uses LLMs to assist at each step. Evans himself [18] has publicly encouraged DDD practitioners to experiment with LLMs for strategic modelling. Industry-oriented writeups [19] describe using LLMs to accelerate ubiquitous-language definition and context-map drafting.

The unifying characteristic of this prior work is that it targets the *design* side of DDD: LLMs are used as collaborative sparring partners that help human experts *produce* a domain model in the first place. None of these works address what happens *after* that domain model has been produced—how to keep an evolving codebase consistent with it. DDD-Enforcer addresses this complementary enforcement problem: given a rulebook (however it was produced), can an LLM-based system reliably flag code that deviates from it?

2.6 Multi-Agent LLM Systems

Multi-agent architectures for LLM-based systems have gained significant attention, with frameworks such as LangGraph, AutoGen [16], and CrewAI providing orchestration capabilities for complex multi-step workflows. These frameworks demonstrate that decomposing a complex reasoning task into specialized agent roles—each with focused responsibilities and structured handoffs—can improve both quality and reliability of LLM outputs.

DDD-Enforcer’s four-stage pipeline (Scout, Architect, Specialist, Synthesizer) draws from these patterns while specializing for domain extraction. Each agent has a clearly defined role, structured input/output contracts enforced through Pydantic schemas, and specific prompt engineering tailored to its stage of the pipeline.

[PLACEHOLDER: Add 2–3 more references on multi-agent LLM systems in SE contexts.]

2.7 Positioning

Our work complements the above strands of research in several ways. Unlike Ozkan et al. [4], who perform DDD analysis and refactoring entirely manually through expert-driven focus groups and interviews, we automate domain model extraction and violation detection using LLMs. Unlike Abdelfattah et al. [5], who use traditional ML with handcrafted features for semantic clone detection, we leverage LLMs’ pre-trained semantic understanding to reason directly about domain concepts without manual feature engineering. Unlike architecture enforcement tools such as ArchUnit [7], which require manually defined structural rules, our system derives rules automatically from SRS documents and can detect semantic violations beyond structural constraints. Unlike prior LLM-assisted DDD work [17,18], which uses LLMs to *produce* a domain model during design, DDD-Enforcer uses LLMs to *enforce* an already-produced model against live code. Table 1 summarizes these distinctions.

3 System Architecture and Design

This section describes the architecture of DDD-Enforcer. The system consists of three main components: (1) a multi-agent pipeline for automatic domain model generation from SRS documents, (2) a code analysis engine combining AST inspection with LLM-powered semantic analysis, and (3) an embedding-based traceability pipeline linking violations back to their originating requirements. Figure 1 provides an overview.

Table 1 Positioning of DDD-Enforcer relative to existing approaches

Aspect		ArchUnit	Ozkan al.	et al.	Abdelfattah et al.	Prior LLM+DDD	DDD-Enforcer (Ours)
DDD phase	lifecycle	Enforcement	Design refactor	&	Post-hoc clone detection	Design assistance	Enforcement
Automation		Manual rules	Fully manual		Semi-auto (labeled data)	Human-in-the-loop LLM	Fully automated
Semantic understanding	under-	None	Human experts	ex-	WordNet similarity	LLM reasoning	LLM reasoning
Domain model source	model	Developer-defined	Expert groups	focus	N/A	LLM-assisted drafting	Auto-extracted from SRS
Analysis scope		Structural constraints	Single	com-	Cross-service clones	Design artifacts	Per-file DDD violations
Training needed	data	N/A	N/A		27K+ labeled pairs	None (zero-shot)	None (zero-shot)

[PLACEHOLDER: Insert system architecture diagram showing: SRS Document → Multi-Agent Pipeline (Scout → Architect → Specialist → Synthesizer) → Domain Model JSON → Code Analysis Engine (AST Parser + LLM Validator) → Violations → Traceability Pipeline → VS Code Extension]

Fig. 1 Overview of the DDD-Enforcer system architecture

3.1 Multi-Agent Domain Model Generation

The domain model generation pipeline transforms unstructured SRS documents into a structured JSON domain model through four specialized LLM agents, each with clearly defined responsibilities and Pydantic-enforced output schemas.

Scout Agent The Scout agent serves as the entry point, receiving the raw SRS document and extracting domain-relevant content. It processes documents through smart chunking (10,000-character chunks with sentence boundary preservation) to accommodate LLM context window limitations. The Scout identifies and extracts sentences describing business entities, domain rules, workflows, and inter-entity relationships, while filtering out boilerplate content (e.g., formatting instructions, version history, legal notices). The output is a curated set of domain-relevant text passages.

Architect Agent The Architect agent analyzes the Scout’s output to identify distinct bounded contexts within the domain. It groups related domain concepts by examining terminology clusters, entity co-occurrence, and logical ownership boundaries. For each identified context, the Architect provides a name, a description of its responsibility, and a preliminary list of key entities. Typically, this stage identifies 2–8 bounded contexts per project, depending on domain complexity.

Specialist Agent The Specialist agent performs deep analysis of each bounded context identified by the Architect. In a single optimized API call per context, it extracts:

- **Aggregate Roots:** Primary entities that serve as consistency boundaries
- **Entities:** Domain objects with identity, including their attributes and relationships
- **Value Objects:** Immutable objects defined by their attribute values
- **Domain Events:** Significant occurrences that the domain tracks
- **Business Rules:** Constraints and invariants governing the domain
- **Synonyms to Avoid:** Terms that should not be used in place of canonical domain terms
- **Context Dependencies:** Allowed interactions between bounded contexts

Synthesizer Agent The Synthesizer agent merges all context-specific analyses into a cohesive domain model. It resolves duplicate entities that may appear across contexts, ensures naming consistency, validates cross-context dependencies, and produces a final JSON output conforming to a Pydantic-enforced `DomainModel` schema.

Structured Output Enforcement All agent interactions use Pydantic schemas to enforce structured outputs. This ensures that LLM responses conform to expected formats, eliminating parsing failures and providing type safety throughout the pipeline. Listing 1 shows the core response schema for violation detection.

Listing 1 Pydantic schema for violation detection response

```
class Violation(BaseModel):
    type: str          # e.g., "SynonymViolation"
    message: str       # Human-readable description
    location: str      # Class/function name
    severity: str      # "error" or "warning"

class ValidationResponse(BaseModel):
    violations: List[Violation]
    summary: str
```

3.2 Code Analysis Engine

The code analysis engine combines traditional AST inspection with LLM-powered semantic analysis.

AST Parsing A custom `DomainVisitor` class performs AST analysis on source files, extracting structural information including: class definitions with base classes and methods, import statements (both `import` and `from imports`), function definitions with parameters and type hints, variable assignments (for value object violation detection), and function calls (for domain event analysis).

LLM-Powered Violation Detection The structural AST data, combined with the domain model rules from `model.json`, are sent to the LLM for semantic analysis. The LLM evaluates the code against six distinct violation categories:

For each violation type below, we give (i) a precise definition, (ii) a negative example (code that violates the rule) and a positive example (the compliant form), and (iii) the detection signal the LLM relies on—the AST feature extracted by the `DomainVisitor` combined with the rulebook field that grounds the judgment.

V1: Synonym Violation. Definition. A class, method, attribute, or variable uses a term that the rulebook has registered as a `synonym_of` a canonical domain term. *Negative:* `class ClientProfile` in a bounded context whose rulebook declares “Client” as a synonym of the canonical “Customer”. *Positive:* `class CustomerProfile`. *Detection signal:* AST class/attribute/variable names $\times$ the rulebook’s `synonyms` list for the bounded context.

V2: Banned Term Violation. Definition. Code within a bounded context uses a term the rulebook has explicitly forbidden in that context (typically because the term belongs to a different bounded context’s language). *Negative:* a method named `calculate_shipping_fee` inside the Billing context whose rulebook bans shipping-related vocabulary. *Positive:* the method lives in the Shipping context, or is renamed to a billing-local concept. *Detection signal:* AST identifier names $\times$ the rulebook’s `banned_terms` list for the enclosing bounded context.

V3: Naming Convention Violation. Definition. A class, method, or variable name does not align with the ubiquitous language even though it is not a registered synonym or banned term (e.g., a domain concept is referred to by a generic name such as `Data`, `Manager`, or `Handler` that obscures the domain term the rulebook prefers). *Negative:* `class OrderManager` where the rulebook mandates `OrderAggregate` or `OrderService`. *Positive:* the domain-aligned name is used. *Detection signal:* AST class/method names $\times$ the rulebook’s `preferred_terms` and stereotype annotations (entity, value object, aggregate, service).

V4: Context Boundary Violation. Definition. A source file imports, instantiates, or references types from another bounded context without a dependency being declared in the rulebook’s `context_dependencies` graph. *Negative:* `from inventory.models import StockItem` appearing in a file that belongs to the Orders context, where the rulebook does not declare an Orders $\rightarrow$ Inventory dependency. *Positive:* either the dependency is added to the rulebook intentionally, or the interaction is refactored through an anti-corruption layer. *Detection signal:* AST import statements + cross-context symbol resolution $\times$ the rulebook’s `context_dependencies` adjacency list.

V5: Aggregate Boundary Violation. Definition. Code reaches inside an aggregate to read or mutate a child entity directly, bypassing the aggregate root, when the rulebook marks the root as the sole consistency boundary. *Negative:* `order.line_items.append(new_item)` from outside the Order aggregate. *Positive:* `order.add_line_item(new_item)` invoked through the root, which enforces invariants. *Detection signal:* AST attribute-access chains and collection mutations $\times$ the rulebook’s `aggregate_roots` and their declared children.

V6: Domain Event Violation. Definition. An operation that the rulebook identifies as event-producing (e.g., “when an Order is placed, an `OrderPlaced` event must be emitted”) completes without emitting the declared event, or emits it under a different name. *Negative:* an `place_order` method that persists the order but never calls the event publisher. *Positive:* the method persists the order and publishes `OrderPlaced`. *Detection signal:* AST function definitions and their call graphs $\times$ the rulebook’s `domain_events` table mapping operations to required event names.

3.3 Traceability Pipeline (UX Component, Not Evaluated)

For developer UX, DDD-Enforcer ships with an embedding-based traceability pipeline that surfaces the SRS paragraph(s) most likely to correspond to each detected violation, independently of the LLM’s violation analysis. The pipeline uses ChromaDB with the `all-MiniLM-L6-v2` sentence transformer, chunks the SRS at paragraph boundaries (default 1,000 characters, 200-character overlap), and returns the top-3 most relevant chunks for each violation. Crucially, the retrieved chunks are *not* injected into the LLM prompt used for detection—they are presented to the developer after the fact—so that detection quality is not conflated with retrieval quality.

We include the traceability pipeline in the framework and in the released code, but we *do not evaluate its retrieval quality in this paper*. Preliminary inspection indicated that retrieval quality is dominated by the SRS chunking strategy rather than by the detection components we study here; evaluating it would require a separate study design with its own ground truth and would dilute the focus on the research questions around rulebook extraction and violation detection. We therefore treat the traceability pipeline as an engineering feature of the current instantiation and leave its evaluation to future work.

3.4 IDE Integration

DDD-Enforcer is surfaced to developers through a VS Code extension. On save, the extension invokes the backend validator for the changed file, receives a `ValidationResponse` (Listing 1), and renders each violation as a VS Code diagnostic anchored to the offending class, method, or import. Each diagnostic

Table 2 Differences between the conference version of DDD-Enforcer and the version studied in this paper

Aspect	Conference version [24]	Current version (this paper)
Rulebook extraction	Single LLM prompt over the full SRS	Four-stage multi-agent pipeline (Scout, Architect, Specialist, Synthesizer) with Pydantic-enforced handoffs
Rulebook schema	Flat term list per context	Structured JSON with aggregates, value objects, events, synonyms, banned terms, and <code>context_dependencies</code> graph
Code analysis input to LLM	Raw source code	Source code plus structured AST features extracted by <code>DomainVisitor</code>
Violation taxonomy	Synonym + naming + context only (3 types)	Six types (V1–V6), including aggregate-boundary and domain-event violations
Structured output	Free-form text, post-hoc parsed	Pydantic-enforced <code>ValidationResponse</code> schema
Traceability	Not present	Embedding-based retrieval (ChromaDB + MiniLM-L6-v2), decoupled from detection
IDE integration	Command-line report	VS Code extension with inline diagnostics and “View Source” action
Evaluation	Illustrative examples only	LLM-Assisted Human Evaluation across pipelines, models, and domains

carries the violation type, a human-readable message, the rulebook identifier that grounds the judgment, and a “View Source” action that opens the corresponding SRS paragraph(s) returned by the traceability pipeline. **[PLACEHOLDER: AUTHOR TODO: include a screenshot figure of the extension showing (a) an in-editor diagnostic for a Synonym Violation and (b) the “View Source” side panel showing the SRS paragraph. Caption it fig:ide-screenshot.]**

3.5 Differences from the Conference Version

This journal paper extends our earlier conference publication [24] with substantive changes to both the system and the evaluation. Table 2 summarizes the differences.

4 Experimental Design

4.1 Research Questions and Study Structure

We investigate four research questions, structured as a funnel: RQ1 selects the best pipeline architecture, RQ2 uses that winner to select the best LLM, RQ3 uses the winning combination to probe cross-domain generalization, and RQ4 is a controlled recall study on codebases with deliberately seeded violations.

RQ1 (Pipeline Comparison): *Holding the LLM fixed, how do alternative pipeline architectures compare in rulebook quality and DDD violation detection?*

Motivation: The multi-agent pipeline introduces architectural complexity. Before comparing LLM providers, we need to know whether that complexity buys anything over simpler alternatives. We compare three pipelines using the same underlying LLM (Section 4.4): a *Naive* single-call pipeline, a *Retrieval-Augmented* single-call pipeline, and our *Multi-Agent* pipeline.

RQ2 (Model Comparison): *Using the best pipeline from RQ1, how do different LLM providers compare on detection accuracy, latency, and cost?*

Motivation: Once the pipeline is fixed, we probe how much the choice of LLM provider matters. We evaluate Gemini (Google), ChatGPT (OpenAI), Claude (Anthropic), and one open-source local model.

RQ3 (Cross-Domain Generalization): *Using the best pipeline and model combination, how does the framework perform across three SRS domains from distinct application areas?*

Motivation: A proof-of-concept is only interesting if it does not fall apart on the second domain. We keep the number of domains intentionally small (three) to keep the evaluation manageable and to avoid diluting the analysis; we report per-domain precision, recall, and F1 together with domain-specific failure modes.

RQ4 (Synthetic-Violation Recognition): *On example codebases generated from an SRS with deliberately seeded DDD violations, how well does the framework recover the planted violations?*

Motivation: On real-world code we never know the true set of violations. In this RQ we flip the setup: we use a strong LLM to generate reference codebases from each SRS, then we manually seed a known set of violations (synonym misuse, bounded-context crossings, aggregate-boundary breaches). This gives us a controlled recall-floor dataset where every planted violation is ground truth by construction. Section 4.7 describes the seeding protocol.

4.2 Subject SRS Documents and Codebases

For RQ3 (cross-domain generalization) we evaluate on three SRS documents drawn from distinct application domains. We keep the number intentionally small: a proof-of-concept study gains clarity from focused comparison, and adding further domains would dilute the analysis without changing the framing

Table 3 SRS documents and codebases used in the cross-domain evaluation (RQ3). Codebases are either sourced from public repositories or, where no suitable open-source counterpart exists for the SRS, generated from the SRS using a strong LLM and then lightly hand-edited for realism.

Label	Domain	Services		LOC	Codebase on
D1	[PLACEHOLDER: Domain 1 (e.g., E-commerce)]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
D2	[PLACEHOLDER: Domain 2 (e.g., Banking)]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
D3	[PLACEHOLDER: Domain 3 (e.g., Healthcare)]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]

Table 4 LLM providers and models evaluated for RQ2. One model per family is included to keep the cross-family comparison balanced. All other pipeline parameters are held constant.

Model	Provider	Deployment	Approx. Cost
[PLACEHOLDER: Gemini 2.x]	Google	API	[PLACEHOLDER: -]
[PLACEHOLDER: GPT-4.x / GPT-5.x]	OpenAI	API	[PLACEHOLDER: -]
[PLACEHOLDER: Claude Sonnet 4.x]	Anthropic	API	[PLACEHOLDER: -]
[PLACEHOLDER: Open-source (e.g. Qwen-Coder / Llama-3 / DeepSeek)]	Community	Local (quantized)	Compute only

of the results. Table 3 summarizes the SRS documents and their associated codebases.

[PLACEHOLDER: AUTHOR TODO: for each of D1–D3 describe: (i) origin of the SRS document (public repo, course artifact, or authored for this study), (ii) why it was selected, (iii) DDD-relevant characteristics (bounded contexts expected, ubiquitous-language richness), (iv) whether the accompanying codebase is third-party or generated from the SRS. Note any licensing or redistribution constraints.]

4.3 LLM Providers Under Study

For RQ2 (model comparison) we evaluate one representative model per provider, spanning the three major commercial families and one open-source option for local deployment. Table 4 lists the models.

Each model is used as a drop-in replacement for the violation-detection LLM call and for the rulebook-extraction agents in the winning pipeline from RQ1. All prompts, temperature settings (0 throughout), AST features, and rulebook schemas are held constant across models. A stricter, strictly-stronger *Judge* LLM—distinct from all tested models—is used for evaluation (Section 4.5).

4.4 Pipelines Under Comparison

For RQ1 we compare three pipeline architectures, all using the same underlying LLM. They form a ladder from raw prompting to structured extraction:

Pipeline P1: Naive. The LLM receives the full SRS text and the full source file in a single prompt and is asked to list DDD violations in free form. No AST extraction, no rulebook, no structured output. This represents the outcome a developer would get by pasting code and SRS into a general-purpose chat interface.

Pipeline P2: Retrieval-Augmented Single-Call. The SRS is chunked (section-aware, $\sim 1,000$ characters with 200-character overlap) and indexed with `all-MiniLM-L6-v2` embeddings. For each source file, the top- k most relevant SRS chunks (default $k=5$) are retrieved using a query built from the file’s class and method names. The LLM is then given these chunks plus the file’s AST features and source code, and is asked to list violations (Pydantic-enforced output). P2 has retrieval and AST but *no* up-front rulebook.

Pipeline P3: Multi-Agent (Ours). The full four-stage pipeline (Scout, Architect, Specialist, Synthesizer) extracts the rulebook once, up front. Each subsequent per-file validation call then receives the complete rulebook plus the file’s AST features and source code, and returns structured violations.

All three pipelines share the same violation taxonomy (V1–V6), the same Pydantic response schema for parseability (P1’s free-form output is parsed best-effort), the same Judge-based evaluation protocol, and identical temperature settings. By swapping only the pipeline architecture, we isolate what the multi-agent decomposition contributes beyond retrieval and beyond raw prompting.

4.5 Evaluation Protocol: LLM-Assisted Human Evaluation

Constructing an expert-annotated ground truth for DDD violations at the scale this study requires—multiple pipelines, multiple LLM providers, and three domains, all cross-multiplied—is infeasible for our team. Recruiting two or more DDD-fluent independent annotators per domain, resolving disagreements, and reporting Cohen’s κ per configuration would dominate the cost of the study without changing its proof-of-concept framing. We adopt a different evaluation protocol, which we call *LLM-Assisted Human Evaluation*. This protocol is directly inspired by the growing body of work on LLM-as-a-Judge methodologies in software engineering [20, 21, 22], which has shown that strong LLMs, when guided by detailed rubrics, approximate human evaluator agreement on many SE tasks.

Step 1: Judge LLM selection. The Judge LLM is required to be a strictly stronger frontier model than any model in the RQ2 tested set (e.g., a top-tier reasoning model from one of the three commercial families). When possible, the Judge belongs to a *different* model family than the tested model whose output is being evaluated, to minimize intra-family agreement bias; cases where this is not possible (e.g., when the top frontier model shares a family with the tested

model) are flagged and treated as sensitivity checks. The Judge is not used anywhere in the pipelines themselves.

Step 2: Judge rubric prompt. The Judge receives a long, structured rubric prompt containing: (a) formal definitions of all six violation types (copied from Section 3.2), (b) the SRS document or the extracted rulebook, (c) the source file under analysis, and (d) the tested system’s reported violations for that file. The Judge is asked to (i) classify each reported violation as a True Positive or False Positive, justifying each verdict by citing the SRS or rulebook passage that supports it, and (ii) list additional violations that the tested system missed (False Negatives), again with citations. The full rubric prompt is released with our replication package.

Step 3: Human audit of Judge verdicts. The authors audit a random sample (plus all cases where the Judge’s confidence marker is low) of the Judge’s TP/FP/FN classifications. For each audited case we record whether the authors agree with the Judge and, on disagreement, we override the Judge with the human verdict. We report three numbers per configuration: the raw Judge-assigned precision/recall/F1, the human-audited precision/recall/F1, and the *author–Judge agreement rate*—the fraction of audited verdicts where the author and the Judge agreed. The agreement rate serves the same role in our protocol that Cohen’s κ serves in traditional ground-truth studies: it is a reliability indicator for the derived labels.

Why this protocol fits a proof-of-concept. The protocol deliberately does not claim to reproduce the quality of a fully expert-annotated gold standard. What it does claim is to be (a) transparent (the rubric prompt and all Judge verdicts are released), (b) reproducible (any future researcher can run the same rubric against a new pipeline or model), and (c) honest about its own limits (the agreement rate is reported explicitly, and human audit overrides the Judge when they disagree). These properties align with the proof-of-concept framing of the paper.

4.6 Evaluation Metrics

We measure the following metrics. Violation detection quality is the primary outcome; the other metrics support interpretation.

Violation Detection Quality.

- **Precision:** $\frac{TP}{TP+FP}$ —the fraction of reported violations judged to be true violations.
- **Recall:** $\frac{TP}{TP+FN}$ —the fraction of judged-true violations that the system reported.
- **F1-score:** $\frac{2 \cdot P \cdot R}{P+R}$ —harmonic mean of precision and recall.
- Computed per violation type (V1–V6) and aggregated across all types, for each configuration.

Rulebook Quality.

- **Entity coverage:** fraction of domain entities mentioned in the SRS that appear in the extracted rulebook.
- **Entity precision:** fraction of rulebook entities that are Judge-confirmed to correspond to SRS concepts (as opposed to hallucinated entries).
- **Bounded-context agreement:** whether the set and names of bounded contexts in the rulebook match those the Judge identifies from the SRS.

Evaluation Reliability.

- **Author–Judge agreement rate:** fraction of audited verdicts where the author agreed with the Judge’s TP/FP/FN classification (replaces Cohen’s κ ; see Section 4.5).

Performance and Cost.

- **Validation latency:** wall-clock time for a single per-file validation call (seconds).
- **Rulebook generation time:** wall-clock time for the full rulebook-extraction pipeline end-to-end (seconds).
- **Cost per validation:** API cost per file analysis (USD), computed from token usage and provider price sheets.

Traceability retrieval quality is intentionally omitted (see Section 3.3).

4.7 Synthetic-Violation Protocol (RQ4)

For RQ4 we construct a controlled recall-floor dataset in three steps.

Step 1: Reference codebase generation. For each of the three SRS documents we use a strong, non-tested LLM (distinct from both the tested models and the Judge) to generate a reference Python microservice codebase that, to the best of the generator’s ability, complies with the SRS. The generator is prompted with the full SRS and the list of violation types (V1–V6) with the instruction to *avoid* all of them. Authors then perform a light cleanup pass to fix syntactic issues and ensure that the generated code typechecks and runs.

Step 2: Deliberate violation seeding. Authors then inject a fixed, catalogued set of violations into the cleaned reference codebase. Each injection is logged with: (a) the file and line edited, (b) the target violation type (one of V1–V6), (c) the edit (e.g., “renamed `CustomerProfile` to `ClientProfile`” or “added cross-context import from `inventory`”), and (d) a short rationale tying the edit to the relevant rulebook entry. Seed counts per violation type and per domain are chosen up front and fixed before any detection runs.

Table 5 Pipeline comparison under a fixed LLM. All three pipelines use the same underlying model, prompts, and Judge configuration. P1 has no rulebook; P2 constructs retrieval context on-the-fly per file; P3 extracts a rulebook up front.

Pipeline	P		R		F1	Parseable %
P1 Naive	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
P2 RAG single-call	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
P3 Multi-Agent (ours)	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]

Step 3: Detection and scoring. The full framework (with RQ1+RQ2 winning configuration) is run on the seeded codebase. For each seeded violation we record whether it was detected (at any granularity that the authors judge acceptable, e.g., right file + right type). Additional violations the framework reports that do not correspond to a seed are triaged by the Judge as in Section 4.5. Primary metric: **seeded-recall** = (#seeded violations detected) / (#seeded violations). This number serves as a recall floor in a setting where the ground truth is completely known.

4.8 Reproducibility, Configuration, and Output Labelling

Every experiment in this study is driven by a single YAML configuration file that specifies: the pipeline (P1/P2/P3), the tested model (provider + model identifier), the Judge model, the SRS document, any seeded-violation manifest (for RQ4), and random seeds. Each run writes its outputs to a dedicated directory named `results/{pipeline}_{model}_{srs}_{run_id}/`, under which we store: (i) the raw LLM request and response for every call, (ii) the parsed `ValidationResponse`, (iii) the Judge verdicts and human-audit overrides, (iv) token counts and timing, and (v) the configuration file that produced the run. The directory naming scheme makes all cross-RQ tables reconstructible by filename alone. The configuration files, seeded-violation manifests, rubric prompts, and raw results are released as part of the replication package.

5 RQ1: Pipeline Comparison

We first compare the three pipelines (P1 Naive, P2 RAG single-call, P3 Multi-Agent; Section 4.4) on one SRS and one held-out LLM, and use LLM-Assisted Human Evaluation to score each pipeline’s reported violations. Table 5 aggregates precision, recall, and F1 across all six violation types; it also reports rulebook quality (for P3, which produces an explicit rulebook), parseable-output rate, and wall-clock cost.

[PLACEHOLDER: AUTHOR TODO (RQ1 analysis): discuss (i) where P3 outperforms P2 and whether the gap is concentrated in particular violation types (we expect it in V1 Synonym and V4 Context-boundary because these depend on global SRS knowledge),

Table 6 Detection quality, latency, and cost by LLM provider, using the RQ1 winning pipeline on a held-out SRS. All numbers are produced via LLM-Assisted Human Evaluation with a strictly stronger Judge model.

Model	P	R	F1	A
[PLACEHOLDER: Gemini 2.x]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
[PLACEHOLDER: GPT-4.x / GPT-5.x]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
[PLACEHOLDER: Claude Sonnet 4.x]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
[PLACEHOLDER: Open-source (local)]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]

[PLACEHOLDER: Figure: Scatter plot with cost/validation on x-axis (log scale), F1 on y-axis, one point per tested model. Annotate the Pareto frontier. Include the open-source model with compute cost approximated to USD.]

Fig. 2 Cost–accuracy Pareto frontier across LLM providers (RQ2). Caption will be filled once data is in.

(ii) where P2 is already “good enough” and therefore whether practitioners without the time to build a full rulebook have a viable fallback, (iii) how often P1 emits unparseable output, and (iv) qualitative examples contrasting a P1 hallucinated violation against a P3 rulebook-grounded one.]

Summary of findings (RQ1). [PLACEHOLDER: Once numbers are in: state which pipeline best balances detection quality and cost, frame it cautiously (“the multi-agent pipeline suggests the largest gains in *these* violation types under *this* LLM”), and carry the winner forward to RQ2.]

6 RQ2: LLM Provider Comparison

Using the RQ1 winning pipeline, we swap in each of the four LLM providers (Gemini, ChatGPT, Claude, one open-source local model) and measure detection quality, latency, and cost. Table 6 reports aggregate metrics; Figure 2 plots the cost–accuracy Pareto frontier.

[PLACEHOLDER: AUTHOR TODO (RQ2 analysis): discuss (i) whether the top-F1 model dominates on all six violation types or whether different models win different types, (ii) whether the open-source model is usable for practitioners who cannot send SRS content to third-party APIs (a common enterprise constraint), (iii) where the cost–accuracy Pareto frontier sits.]

Summary of findings (RQ2). [PLACEHOLDER: Once numbers are in: state the best model under the current setup, frame cautiously (“at today’s capability level, *this* model appears to be the most cost-effective; this ranking will shift as providers release new versions”), and carry the winner forward to RQ3 and RQ4.]

Table 7 Per-domain detection quality using the RQ1+RQ2 winning configuration.

Domain	Violations (Judge)		TP	Precision
D1 [PLACEHOLDER: (domain name)]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
D2 [PLACEHOLDER: (domain name)]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
D3 [PLACEHOLDER: (domain name)]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]

Table 8 Seeded-violation recognition using the RQ1+RQ2 winning configuration. Seeds are known by construction; detection is judged right-file + right-type.

Violation type	Seeded	Detected	Seeded-recall
V1 Synonym	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
V2 Banned Term	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
V3 Naming Convention	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
V4 Context Boundary	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
V5 Aggregate Boundary	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
V6 Domain Event	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]
Overall	[PLACEHOLDER: -]	[PLACEHOLDER: -]	[PLACEHOLDER: -]

7 RQ3: Cross-Domain Generalization

Using the RQ1+RQ2 winning configuration, we evaluate on three SRS documents drawn from distinct domains (D1, D2, D3). Table 7 reports per-domain precision, recall, and F1; per-violation-type breakdowns are in the replication package.

[PLACEHOLDER: AUTHOR TODO (RQ3 analysis): identify (i) which domain is hardest and *why* (e.g., vocabulary overlap between contexts, sparser SRS, more tightly coupled domains), (ii) the most common failure modes per domain, (iii) whether any violation type degrades consistently across all domains (that would be a framework-level weakness as opposed to a domain-specific one).]

Summary of findings (RQ3). [PLACEHOLDER: Once numbers are in: comment cautiously on generalization, flag any domain where the framework is noticeably weaker, and be explicit that three domains is a small sample intended to probe generalization, not to establish it.]

8 RQ4: Synthetic-Violation Recognition

For RQ4 we use the generated reference codebases with deliberately seeded violations (Section 4.7) to measure the framework’s recall against a ground truth that we fully control. Table 8 reports, per violation type, the number of violations seeded, the number detected, and the resulting seeded-recall.

[PLACEHOLDER: AUTHOR TODO (RQ4 analysis): discuss (i) which violation types the framework is most and least reliable at recovering, (ii) whether any *non-seeded* additional violations the

framework reported were confirmed by the Judge as real (an interesting bonus-signal), and (iii) whether recall is dominated by missed violations or by violations detected under a different type than they were seeded (type-confusion).]

Summary of findings (RQ4). [PLACEHOLDER: Once numbers are in: report seeded-recall per violation type, flag the weakest type, and frame the RQ4 result as a controlled-setting recall floor rather than a generalizable recall estimate.]

9 Discussion

9.1 Key Findings and Implications

We set out not to prove that DDD-Enforcer is a finished product, but to demonstrate that LLM-enforced DDD is feasible at today’s capability level and to release a framework that others can extend. Read against that goal, our results make four concrete contributions.

A framework, not a point solution. The separation between rulebook extraction (P3) and violation detection (AST + rulebook + LLM) lets future researchers swap pipelines, models, and even violation taxonomies independently. The three pipelines studied in RQ1 are instances of the same framework, not competing products.

Multi-agent decomposition pays off where global SRS knowledge matters. [PLACEHOLDER: Fill once RQ1 numbers are in: describe in which violation types P3 pulls ahead of P2 and where P2 is already competitive. Be explicit that this is a statement about today’s LLMs: as context windows and reasoning improve, the gap between P2 and P3 may shrink, and the framework is designed to let future researchers re-measure it.]

Provider choice matters less than one might expect. [PLACEHOLDER: Fill once RQ2 numbers are in: describe the spread between providers and whether the open-source local model is close enough to be practical for enterprises with data-egress constraints.]

The rulebook is a durable artifact. Even when detection is imperfect, the rulebook produced by the multi-agent pipeline is a human-readable, machine-checkable summary of the project’s ubiquitous language and bounded contexts. For teams onboarding new developers, this is useful independently of the accuracy of automated enforcement: it is a shared reference that was previously locked up in tribal knowledge.

Caveats. All headline numbers in this paper come from LLM-Assisted Human Evaluation, not from expert-annotated ground truth. The author–Judge agreement rate [PLACEHOLDER: (report final value)] indicates [PLACEHOLDER: (characterize reliability)], and we flag throughout the paper that these numbers should be read as proof-of-concept evidence rather than as production benchmarks.

9.2 Comparison with Related Approaches

Our findings extend the landscape of architecture enforcement research in several ways.

Compared to the manual DDD refactoring approach of Ozkan et al. [4], which required embedded researchers conducting focus groups and interviews over an extended period, DDD-Enforcer automates domain model extraction and violation detection. This comparison highlights that our tool addresses a different but complementary need: Ozkan et al. focus on *performing* DDD refactoring, while we focus on *continuously enforcing* DDD compliance. Their study demonstrated the value of DDD through practitioner perception (mTAM score of 84/100); our work complements this by providing automated, continuous enforcement that can sustain DDD compliance after initial refactoring.

Compared to Abdelfattah et al.’s [5] semantic clone detection ($F1 = 0.813$), our approach addresses a complementary problem. Semantic clones are a symptom of DDD violations—duplicated functionality across services often indicates that bounded context boundaries were drawn incorrectly. Our violation detection operates upstream, identifying the design-level issues that may eventually lead to semantic clones.

Compared to prior LLM-assisted DDD work [17,18], which uses LLMs as sparring partners for *designing* a domain model, DDD-Enforcer addresses what happens after the model has been designed: how to keep the codebase consistent with it as it evolves. These two strands are complementary—an LLM that helps produce a rulebook and an LLM that enforces it can in principle be chained end-to-end, and the framework described in this paper is explicitly designed to accept a rulebook from either source.

9.3 Threats to Validity

We organize threats to validity around the three LLM-specific risks that most directly shape our study (context-window limits, hallucinations, evaluation subjectivity), followed by traditional internal, external, and construct threats. For each threat we describe the design choice that mitigates it and the residual risk that remains.

9.3.1 Context-window limitations

LLM context windows are finite, and both SRS documents and source files can in principle exceed them. *Mitigation.* The Scout stage chunks the SRS into $\sim 10,000$ -character passages with sentence-boundary preservation so that no prompt ever carries the full SRS directly. The validator does not send raw source code: it sends AST-extracted features (class, method, import, and variable summaries) plus a targeted source slice, which compresses per-file input substantially. Rulebooks are kept as structured JSON so that the per-file validation prompt only needs the rulebook plus the single file under analysis.

Residual risk. Very large monolithic source files may still exceed what the validator can analyze in one call. We document, for each tested configuration, the size at which truncation begins and note the affected files in the raw results.

9.3.2 Hallucinations

LLMs can invent violations that are not grounded in the rulebook, or miss real violations while confidently producing plausible-looking output. *Mitigation.* Structured outputs (Pydantic schemas, Listing 1) require every reported violation to carry a `rulebook_id` field; reports that do not match a rulebook entry are discarded as ungrounded. The LLM-Assisted Human Evaluation protocol (Section 4.5) classifies hallucinated violations as false positives and surfaces them in the precision metric. The Judge LLM is also asked to flag apparent hallucinations explicitly in its rubric. *Residual risk.* When the Judge and the tested model belong to the same family, they may share correlated hallucination patterns and agree on mistakes. We address this by (a) preferring cross-family Judges where the top-of-family rule permits it and (b) flagging intra-family Judge/tested pairs as sensitivity checks rather than headline numbers; the audit pass then manually inspects those cases.

9.3.3 Evaluation subjectivity

Without an expert-annotated ground truth, our headline precision/recall/F1 numbers depend on a Judge LLM and on the authors’ audit of its verdicts. *Mitigation.* We adopt the LLM-Assisted Human Evaluation protocol (Section 4.5) to make the subjectivity explicit: the rubric prompt is released, Judge verdicts are released, authors audit a random sample plus all low-confidence cases, and we report the author–Judge agreement rate as a reliability indicator. Cohen’s κ is not applicable because there is no second independent annotator; the agreement rate serves the analogous role. *Residual risk.* If the Judge systematically misjudges borderline cases in a direction the authors share, the audit will not catch the bias. We treat the reported numbers as proof-of-concept evidence, not as production benchmarks, and release all raw data so that future work can re-score with a different Judge or with human experts.

9.3.4 Internal validity

LLM non-determinism. Even at temperature 0, some providers introduce small amounts of output variation. We set temperature to 0 throughout and report results averaged over **[PLACEHOLDER: N]** independent runs per configuration; variance across runs is reported in the replication package.

Prompt sensitivity. The pipelines and the Judge rubric are driven by handcrafted prompts; we did not perform systematic prompt optimization. Different prompt formulations may yield different numbers. Our prompts are released so that future work can quantify prompt sensitivity.

9.3.5 External validity

Language limitation. The current `DomainVisitor` supports only Python source files. The LLM-based semantic analysis is language-agnostic in principle, but the AST extraction component must be re-implemented for Java, TypeScript, C#, and Go before industrial microservice systems—many of which are Java/Spring—can be evaluated directly.

Domain count. RQ3 uses three SRS domains. This is sufficient to probe whether the framework generalizes beyond a single domain, but not sufficient to establish generalization. We are explicit throughout the paper that three domains is a probe, not a benchmark.

SRS availability. Real-world projects rarely have comprehensive, up-to-date SRS documents. Where no public SRS existed for a domain we studied, we authored one based on available documentation. The quality of authored SRS documents may differ from what would be available in industrial settings.

9.3.6 Construct validity

Violation taxonomy. Our six violation types (V1–V6) are one operationalization of DDD compliance. Other researchers may define additional types (e.g., service-layer leakage, eventual-consistency violations) that our framework does not measure. The framework is designed to accept new types without structural changes, but results in this paper are scoped to V1–V6.

Pipeline design choices. P1, P2, and P3 are specific instantiations of “naive”, “retrieval-augmented”, and “multi-agent” ideas. Stronger naive prompting (few-shot with DDD exemplars) or stronger retrieval (hybrid BM25 + dense) might narrow the gaps we report. Our pipelines represent reasonable starting points, not optimized endpoints.

9.3.7 Conclusion validity

[PLACEHOLDER: AUTHOR TODO: once the number of Judge-labelled violations per configuration is known, discuss whether the sample sizes are large enough to support the precision/recall estimates or whether they should be reported as indicative ranges rather than point estimates.]

10 Conclusion

In this paper, we presented DDD-Enforcer as a *proof-of-concept framework* for LLM-powered Domain-Driven Design enforcement. Rather than arguing for a single finished system, we aimed to demonstrate that LLM-enforced DDD is feasible with today’s models and to provide a reusable scaffolding—with swappable pipelines, models, and violation taxonomies—that other researchers can

extend. Across four research questions (pipeline comparison, provider comparison, cross-domain generalization, and synthetic-violation recognition), we evaluated an instance of the framework using LLM-Assisted Human Evaluation and released all configurations, prompts, Judge verdicts, and raw results.

Read as a proof of concept, our study suggests that:

- [PLACEHOLDER: RQ1: the multi-agent pipeline (P3) improves F1 over simpler alternatives in violation types that depend on global SRS knowledge, while retrieval-augmented single-call (P2) is a reasonable fallback for teams that cannot afford the full rulebook-extraction pass.]
- [PLACEHOLDER: RQ2: among Gemini, ChatGPT, Claude, and the open-source local model we tested, the cost–accuracy Pareto frontier at today’s prices favors *[winner]*; the open-source model is *[close enough / not yet close enough]* to be a practical choice for teams with data-egress constraints.]
- [PLACEHOLDER: RQ3: the framework generalizes across the three domains we studied to within *[range]* F1, with *[domain]* being the hardest due to *[reason]*.]
- [PLACEHOLDER: RQ4: on deliberately seeded codebases, the framework recovers *[fraction]* of planted violations; the weakest type is *[type]*, which suggests a concrete target for future prompt or taxonomy improvements.]

We do not claim that DDD-Enforcer is a finished product or that these numbers will still hold as LLMs improve. What we claim is that (i) the framework is useful as-is, including the rulebook as a standalone onboarding artifact, (ii) LLM-Assisted Human Evaluation is a practical methodology for proof-of-concept studies where expert annotation is infeasible, and (iii) the open release invites others to plug in new pipelines, providers, and domains and re-run the study.

Future Work We identify several directions:

- **Multi-language support:** extending `DomainVisitor` to Java, TypeScript, C#, and Go to cover the most common microservice implementation languages.
- **Automated fix suggestions:** generating code transformations that resolve detected violations.
- **Rulebook drift over time:** longitudinal study of how rulebook quality evolves as the SRS itself changes, and how the framework detects rulebook staleness.
- **Beyond DDD:** extending the framework to other architectural styles with explicit rule sets (hexagonal, CQRS, event-sourcing).
- **Graph-based domain representation:** using knowledge graphs to capture cross-context relationships and enable richer reasoning about aggregate and context boundaries.

- **Prompt sensitivity study:** systematic exploration of how Judge rubric formulations shift the measured agreement rate.

Data Availability DDD-Enforcer is available as open-source software at <https://github.com/barandincoguz/DDD-Enforcer>. **[PLACEHOLDER: AUTHOR TODO: add links to the replication package, configuration files, Judge rubric prompts, Judge verdicts, author-audit overrides, seeded-violation manifests for RQ4, and raw per-run result directories.]**

Declarations

Conflict of Interest The authors declare that they have no conflict of interest.

Author Contributions **[PLACEHOLDER: Describe each author’s contributions.]**

References

1. Evans E (2003) Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional
2. Vernon V (2013) Implementing Domain-Driven Design. Addison-Wesley Professional
3. Microsoft (2024) Using tactical DDD to design microservices. Microsoft Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/tactical-ddd>
4. Ozkan O, Babur O, van den Brand M (2023) Refactoring with domain-driven design in an industrial context: An action research report. Empirical Software Engineering 28:94. <https://doi.org/10.1007/s10664-023-10310-1>
5. Abdelfattah AS, Rodriguez A, Walker A, Cerny T (2023) Detecting Semantic Clones in Microservices Using Components. SN Computer Science 4:470. <https://doi.org/10.1007/s42979-023-01910-1>
6. Loukides M, Swoyer S (2020) Microservices Adoption in 2020. O’Reilly Media
7. ArchUnit (2024) ArchUnit – Unit test your Java architecture. <https://www.archunit.org/>
8. NDepend (2024) NDepend – .NET static analysis tool. <https://www.ndepend.com/>
9. Hou X, Zhao Y, Liu Y, et al (2024) Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology 33(3):1–79
10. Fan A, Gokkaya B, Harman M, et al (2023) Large Language Models for Software Engineering: Survey and Open Problems. In: Proceedings of the IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)
11. Chen M, Tworek J, Jun H, et al (2021) Evaluating Large Language Models Trained on Code. arXiv:2107.03374
12. **[PLACEHOLDER: Add reference: LLM-based bug detection paper]**
13. Li L, et al (2022) CodeReviewer: Pre-Training for Automating Code Review Activities. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)
14. **[PLACEHOLDER: Add reference: LLM for program comprehension]**
15. Yetiştiren B, Özsoy I, Tüzün E (2023) Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778
16. Wu Q, Bansal G, Zhang J, et al (2023) AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155

17. **[PLACEHOLDER: AUTHOR TODO: confirm author list and venue.]** (2024) Automating Domain-Driven Design: Experience with a Prompting Framework. arXiv:2603.26244. <https://arxiv.org/abs/2603.26244>
18. Evans E (2024) Eric Evans Encourages DDD Practitioners to Experiment with LLMs. InfoQ. <https://www.infoq.com/news/2024/03/Evans-ddd-experiment-llm/>
19. **[PLACEHOLDER: AUTHOR TODO: pick a specific industry writeup (e.g., Mülder 2024 on LLM-constrained DSL generation, or Sebastian 2024 “Enhancing Domain-Driven Design with Generative AI”) and cite it concretely.]**
20. Wei J, et al (2025) Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. In: Proceedings of ISSTA 2025. arXiv:2502.06193. <https://arxiv.org/abs/2502.06193>
21. Gu J, et al (2024) A Survey on LLM-as-a-Judge. arXiv:2411.15594. <https://arxiv.org/abs/2411.15594>
22. Li H, et al (2024) LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. arXiv:2412.05579. <https://arxiv.org/abs/2412.05579>
23. **[PLACEHOLDER: AUTHOR TODO: confirm author list.]** (2026) An evaluation study of large language models for addressing code quality issues. Empirical Software Engineering. <https://doi.org/10.1007/s10664-026-10858-8>
24. Dincoguz AB, Kendir A, Karakaya M (2025) DDD-Enforcer: An AI-Powered Multi-Agent System for Real-Time Domain-Driven Design Enforcement. **[PLACEHOLDER: Conference name and proceedings details]**